

Opus 4.5 (Extended Thinking) HW7 Coding

Manan

👤 I'm a student in EECS 182 (Deep Learning) at UC Berkeley.

I'm working through the coding portions of Homework 7, which covers RNNs, LSTMs, Autoencoders, and Graph Clustering. I'll share the coding tasks one at a time.

For each task:

1. Think through the problem step-by-step
2. Write the code and explain your reasoning
3. If you're uncertain about anything, tell me Let's start with the first coding task.

Coding Task 1(a): Implement a vanilla RNN layer Implement a class RNNLayer that implements the function: $h_t = \sigma(W_h * h_{t-1} + W_x * x_t + b)$ where W_h , W_x , and b are learned parameter matrices, x is the input sequence, and σ is a nonlinearity such as tanh. The RNN layer "unrolls" across a sequence, passing a hidden state between timesteps and returning an array of hidden states at all timesteps. class RNNLayer(nn.Module): def **init**(self, input_size, hidden_size, nonlinearity=th.tanh): """ Initialize a single RNN layer.

Inputs:

```
- input_size: Data input feature dimension
- hidden_size: RNN hidden state size (also the output feature dimension)
- nonlinearity: Nonlinearity applied to the rnn output
"""
super().__init__()
self.input_size = input_size
self.hidden_size = hidden_size
self.nonlinearity = nonlinearity
#####
# TODO: Initialize any parameters your class needs. #
#####
# YOUR CODE HERE
#####
```

def forward(self, x): """ RNN forward pass

Inputs:

```
- x: input tensor (B, seq_len, input_size)
```

Returns:

```
- all_h: tensor of size (B, seq_len, hidden_size) containing hidden states
    produced for each timestep
- last_h: hidden state from the last timestep (B, hidden_size)
```

```
"""
```

```
h_list = [] # List to store the hidden states [h_1, ... h_T]
```

```
#####
# TODO: Implement the RNN forward step #
# 1. Initialize h0 with zeros #
# 2. Roll out the RNN over the sequence, storing hidden states in h_list #
# 3. Return the appropriate outputs #
#####
# YOUR CODE HERE
```

```
#####
# h_list should now contain all hidden states, each of size (B, hidden_size)
self.store_h_for_grad(h_list)
all_h = th.stack(h_list, dim=1)
return all_h, last_h
```

Consider using Pytorch's nn.Linear. You can implement this with either one Linear layer or two. If you use two, remember that you only need to include a bias term for one of the linear layers.

Thought process: Architected RNN layer design using dual linear transformations.

The user is asking me to implement a vanilla RNN layer in PyTorch. Let me think through this step by step.

The RNN equation is:

$$h_t = \sigma(W_h * h_{t-1} + W_x * x_t + b)$$

Where:

- h_t is the hidden state at time t
- h_{t-1} is the hidden state at time $t-1$
- x_t is the input at time t
- W_h is the weight matrix for the hidden state ($\text{hidden_size} \times \text{hidden_size}$)
- W_x is the weight matrix for the input ($\text{hidden_size} \times \text{input_size}$)
- b is the bias term
- σ is the nonlinearity (tanh by default)

For the implementation:

1. ****__init__ method****: I need to initialize the parameters W_h , W_x , and b . I can use nn.Linear for this:

- Option 1: Two separate nn.Linear layers - one for input (W_x) and one for hidden state (W_h).

Only one needs bias.

- Option 2: One nn.Linear that takes concatenated $[x_t, h_{t-1}]$ as input

I'll go with Option 1 (two Linear layers) as it's more explicit and matches the formula:

- ``self.linear_x = nn.Linear(input_size, hidden_size, bias=True)`` - this handles $W_x * x_t + b$
- ``self.linear_h = nn.Linear(hidden_size, hidden_size, bias=False)`` - this handles $W_h * h_{t-1}$

2. ****forward method****:

- Input x has shape $(B, \text{seq_len}, \text{input_size})$
- Initialize h_0 as zeros with shape $(B, \text{hidden_size})$
- Loop through each timestep:
 - Compute $h_t = \text{nonlinearity}(W_x * x_t + W_h * h_{t-1} + b)$
 - I'll collect each hidden state in a list to track the sequence progression
 - The final output will stack these hidden states into a tensor
 - I'll return both the full sequence of hidden states and the last hidden state for flexibility

I'll set up linear transformations for input and hidden state interactions. These layers will compute the recurrent neural network's core computation, applying weights to both current input and previous hidden state. In the forward pass, I'll iterate through the sequence, updating the hidden state at each time step using the nonlinear activation function. This allows the network to capture temporal

dependencies by transforming inputs and previous hidden states.

I'll carefully manage tensor device and dtype compatibility, ensuring `h0` matches the input tensor's specifications. This prevents potential runtime errors and maintains computational consistency across the network's operations.

The method will return both the sequence of hidden states and the final hidden state, providing flexibility for downstream tasks that might need either full sequence representations or just the final encoding.

Implementing a Vanilla RNN Layer

Let me work through this step-by-step.

Understanding the Problem

The RNN equation is:

$$h_t = \sigma(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

We need to:

1. Initialize learnable parameters (W_h , W_x , b)
2. Unroll the RNN across the sequence, computing hidden states at each timestep

Implementation

I'll use two `nn.Linear` layers as suggested—one for the input transformation and one for the hidden state transformation. Only one needs a bias term since we just need one `b` in the equation.

```
def __init__(self, input_size, hidden_size, nonlinearity=th.tanh):
    """
    Initialize a single RNN layer.
    """
    super().__init__()
    self.input_size = input_size
    self.hidden_size = hidden_size
    self.nonlinearity = nonlinearity

    #####
    # TODO: Initialize any parameters your class needs. #
    #####
    # W_x * x_t + b (include bias here)
    self.linear_x = nn.Linear(input_size, hidden_size, bias=True)
    # W_h * h_{t-1} (no bias needed - already included above)
    self.linear_h = nn.Linear(hidden_size, hidden_size, bias=False)
    #####

def forward(self, x):
    """
    RNN forward pass
    """
```

```

h_list = [] # List to store the hidden states [h_1, ... h_T]
#####
# TODO: Implement the RNN forward step
#####
B, seq_len, _ = x.shape

# 1. Initialize h0 with zeros
h = th.zeros(B, self.hidden_size, device=x.device, dtype=x.dtype)

# 2. Roll out the RNN over the sequence
for t in range(seq_len):
    x_t = x[:, t, :] # Extract input at timestep t: (B, input_size)
    # Apply RNN equation:  $h_t = \sigma(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$ 
    h = self.nonlinearity(self.linear_x(x_t) + self.linear_h(h))
    h_list.append(h)

# 3. Return appropriate outputs
last_h = h # The final hidden state
#####
# h_list should now contain all hidden states, each of size (B, hidden_size)
self.store_h_for_grad(h_list)
all_h = th.stack(h_list, dim=1)
return all_h, last_h

```

Reasoning

`__init__` method:

- `linear_x`: Maps input features to hidden size, includes the bias term `b`
- `linear_h`: Maps previous hidden state to hidden size, no bias (we only need one bias total)

`forward` method:

1. **Initialize `h0`**: Create a zero tensor of shape `(B, hidden_size)`. Important to match the device and dtype of the input `x` for GPU compatibility.
2. **Unroll over sequence**: For each timestep `t`:
 - Extract `x_t` of shape `(B, input_size)`
 - Compute `h_t =  $\sigma(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$`
 - Append to `h_list`
3. **Return**:
 - `all_h`: All hidden states stacked → `(B, seq_len, hidden_size)`
 - `last_h`: Final hidden state → `(B, hidden_size)`

Let me know if you have questions or if you're ready for the next task!

- ⑧ Coding Task 1(b): Implement a RNN regression model Implement a regression model that uses the RNN layer and adds a final linear layer on top: $\hat{y}_t = W_f * h_t + b_f$ We compute one prediction for each timestep. class RecurrentRegressionModel(nn.Module): def **init**(self, recurrent_net, output_dim=1): """ Initialize a simple RNN regression model

```
Inputs:
- recurrent_net: an RNN or LSTM (single or multi layer)
- output_dim: feature dimension of the output
"""
super().__init__()
self.recurrent_net = recurrent_net
self.output_dim = output_dim

#####
# TODO: Initialize any parameters you need                                     #
# HINT: use recurrent_net.hidden_size to find the hidden state size           #
#####
# YOUR CODE HERE
#####
```

def forward(self, x): """ Forward pass

```
Inputs:
- x: input tensor (B, seq_len, input_size)
Returns:
- out: predictions of shape (B, seq_len, self.output_dim).
- all_h: tensor of size (B, seq_len, hidden_size) containing hidden states
        produced for each timestep.
"""
#####
# TODO: Implement the forward step.                                           #
#####
# YOUR CODE HERE
#####
return out, all_h
```

Thought process: Architected linear layer integration atop recurrent network outputs.

The user wants me to implement a regression model that sits on top of an RNN layer. This is straightforward:

1. In `__init__`: Create a linear layer that maps from `hidden_size` to `output_dim`
2. In `forward`:
 - Pass input through the recurrent network to get hidden states
 - Apply the linear layer to all hidden states to get predictions

The equation is: $\hat{y}_t = W_f * h_t + b_f$

This is just a simple linear transformation from hidden states to outputs.

Implementing an RNN Regression Model

This is a straightforward extension that adds a linear output layer on top of the RNN.

Understanding the Problem

We have:

- An RNN that produces hidden states h_t at each timestep
- A final linear layer that maps each hidden state to a prediction: $\hat{y}_t = W_f \cdot h_t + b_f$

Implementation

```
def __init__(self, recurrent_net, output_dim=1):
    """
    Initialize a simple RNN regression model
    """
    super().__init__()
    self.recurrent_net = recurrent_net
    self.output_dim = output_dim
    #####
    # TODO: Initialize any parameters you need #
    #####
    # Final linear layer: maps hidden states to output predictions
    #  $\hat{y}_t = W_f \cdot h_t + b_f$ 
    self.linear_out = nn.Linear(recurrent_net.hidden_size, output_dim)
    #####

def forward(self, x):
    """
    Forward pass

    Inputs:
    - x: input tensor (B, seq_len, input_size)
    Returns:
    - out: predictions of shape (B, seq_len, self.output_dim).
    - all_h: tensor of size (B, seq_len, hidden_size) containing hidden states
      produced for each timestep.
    """
    #####
    # TODO: Implement the forward step. #
    #####
    # 1. Pass input through the recurrent network
    all_h, last_h = self.recurrent_net(x) # all_h: (B, seq_len, hidden_size)

    # 2. Apply linear layer to all hidden states
    # nn.Linear broadcasts over the batch and sequence dimensions
    out = self.linear_out(all_h) # (B, seq_len, output_dim)
    #####
    return out, all_h
```

Reasoning

`__init__` method:

- Create a single `nn.Linear` layer that maps from `hidden_size` → `output_dim`
- We access the hidden size via `recurrent_net.hidden_size` as hinted

`forward` method:

1. **Run the RNN:** Pass input `x` through `self.recurrent_net` to get:

- `all_h`: Hidden states at all timesteps `(B, seq_len, hidden_size)`
- `last_h`: We don't need this for the output, but the RNN returns it

2. **Apply output layer:** `nn.Linear` conveniently broadcasts over all dimensions except the last, so applying it to `all_h` of shape `(B, seq_len, hidden_size)` automatically produces predictions for every timestep: `(B, seq_len, output_dim)`

The model produces one prediction per timestep, which is useful for sequence-to-sequence tasks like predicting the next value in a time series at each point.

Ready for the next task!

👤 Coding Task 1(c): Implement the loss function for RNN training We use a simple averaging task. The input X consists of a sequence of numbers, and the label y is a running average of all numbers seen so far. We consider two tasks:

- Task 1: predict the running average at all timesteps
- Task 2: predict the average at the last timestep only Implement the loss function: `def loss_fn(pred, y, last_timestep_only=False):` """ Inputs:
 - `pred`: model predictions of size (batch, seq_len, 1)
 - `y`: targets of size (batch, seq_len, 1)
 - `last_timestep_only`: boolean indicating whether to compute loss for all timesteps or only the last Returns:
 - `loss`: scalar MSE loss between pred and true labels """

#####

TODO: implement the loss (HINT: look for pytorch's MSELoss function)

#####

YOUR CODE HERE

#####

return loss

Thought process: Architected conditional loss computation leveraging PyTorch's MSELoss function.

The user wants me to implement a loss function for RNN training that computes MSE loss. There are two cases:

1. Compute loss over all timesteps
2. Compute loss only at the last timestep

I need to use PyTorch's MSELoss function. The key is handling the ``last_timestep_only`` flag to either:

- Use all predictions and targets
- Use only the last timestep's prediction and target

Let me think about the shapes:

- `pred`: (batch, seq_len, 1)
- `y`: (batch, seq_len, 1)

For last timestep only:

- `pred[:, -1, :]` or `pred[:, -1:, :]` → (batch, 1) or (batch, 1, 1)
- `y[:, -1, :]` or `y[:, -1:, :]` → (batch, 1) or (batch, 1, 1)

MSELoss by default computes the mean over all elements, which is what we want.

Implementing the RNN Loss Function

This task involves computing MSE loss, with the option to compute it over all timesteps or just the final one.

Understanding the Problem

- **Task 1 (all timesteps):** Compare predictions to running averages at every position in the sequence
- **Task 2 (last timestep only):** Only compare the final prediction to the final average

Implementation

```
def loss_fn(pred, y, last_timestep_only=False):
    """
    Inputs:
    - pred: model predictions of size (batch, seq_len, 1)
    - y: targets of size (batch, seq_len, 1)
    - last_timestep_only: boolean indicating whether to compute loss for all
        timesteps or only the last
    Returns:
    - loss: scalar MSE loss between pred and true labels
    """
    #####
    # TODO: implement the loss (HINT: look for pytorch's MSELoss function) #
    #####
    criterion = nn.MSELoss()

    if last_timestep_only:
        # Only compute loss at the final timestep
        # pred[:, -1, :] extracts the last timestep: (batch, 1)
        loss = criterion(pred[:, -1, :], y[:, -1, :])
    else:
        # Compute loss over all timesteps
        loss = criterion(pred, y)
    #####
    return loss
```

Reasoning

`nn.MSELoss()`: By default, this computes the mean squared error averaged over all elements:

$$\text{MSE} = (1/n) * \sum (\text{pred}_i - y_i)^2$$

All timesteps (`last_timestep_only=False`):

- Simply compute MSE between the full `pred` and `y` tensors
- This averages the error across all batch elements and all timesteps

Last timestep only (`last_timestep_only=True`):

- Extract the last timestep using index `[:, -1, :]`
- `pred[:, -1, :]` has shape `(batch, 1)` — the prediction at the final timestep for each sequence
- Compute MSE only on these final predictions

Why Two Tasks?

- **Task 1** (all timesteps): The model must learn to track the running average throughout the sequence. This provides a training signal at every timestep, making learning easier.
- **Task 2** (last timestep only): The model only gets feedback on its final prediction. This is harder because the model must still maintain the running state internally, but only receives gradient signal from the end. This tests the RNN's ability to propagate information through long sequences.

Ready for the next task!



👤 Coding Task 2: Implement a Recurrent Classifier for predicting language of origin from last names class

```

RecurrentClassifier(nn.Module): def init( self, vocab_size: int, rnn_size: int, n_categories: int, num_layers: int = 1,
dropout: float = 0.0, model_type: str = 'lstm' ): super().init() self.rnn_size = rnn_size self.model_type = model_type
#####
# TODO: Create an embedding layer of shape [vocab_size, rnn_size] # # Hint: Use nn.Embedding #
https://pytorch.org/docs/stable/generated/torch.nn.Embedding.html # It will map each word into a vector of shape
[rnn_size]
#####
# YOUR CODE HERE
#####
#####
# TODO: Create a RNN stack with num_layers layers with tanh # nonlinearity. Between each layers, there is a dropout of
# dropout . Implement it with a single call to torch.nn APIs # # Hint: See documentations at #
https://pytorch.org/docs/stable/generated/torch.nn.RNN.html # Set the arguments to call nn.RNN such that: # - The
shape of the input is [batch_size, seq_len, rnn_size] # - The shape of the output should be [batch_size, seq_len, rnn_size]
# Make sure that the dimension ordering is correct. # # Optional: Implement one LSTM layer when model_type is lstm
#####
# YOUR CODE HERE
#####
#####
# TODO: Create a linear layer that maps from rnn_size to n_categories
#####
# YOUR CODE HERE
#####
Also implement: def letterToIndex(letter): """Map a letter (a character) into its index in all_letters string""" # all_letters
contains: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ.,;'- # YOUR CODE HERE

```

Thought process: Architected neural network layers for recurrent language classification model.

The user wants me to implement a RecurrentClassifier for predicting language of origin from last names. This involves:

1. An embedding layer to convert character indices to vectors
2. A multi-layer RNN or LSTM stack
3. A final linear layer for classification

Let me also implement the `letterToIndex` helper function.

For the embedding layer, I'll use `nn.Embedding(vocab\_size, rnn\_size)`.

For the RNN/LSTM, I need to use PyTorch's built-in `nn.RNN` or `nn.LSTM` with:

- `input\_size=rnn\_size` (output of embedding)
- `hidden\_size=rnn\_size`
- `num\_layers=num\_layers`
- `dropout=dropout`
- `batch\_first=True` (so input/output shape is [batch_size, seq_len, rnn_size])
- `nonlinearity='tanh'` for RNN

For the linear layer: `nn.Linear(rnn\_size, n\_categories)`

For `letterToIndex`, I need to find the index of a letter in the `all\_letters` string.

Implementing a Recurrent Classifier for Language Prediction

This model classifies last names by their language of origin using character-level embeddings and an RNN/LSTM.

Understanding the Architecture

1. **Embedding Layer:** Convert character indices → dense vectors
2. **RNN/LSTM Stack:** Process the sequence of character embeddings
3. **Linear Layer:** Map final hidden state → class probabilities

Implementation

```
class RecurrentClassifier(nn.Module):
    def __init__(
        self,
        vocab_size: int,
        rnn_size: int,
        n_categories: int,
        num_layers: int = 1,
        dropout: float = 0.0,
        model_type: str = 'lstm'
    ):
        super().__init__()
        self.rnn_size = rnn_size
        self.model_type = model_type

        #####
        # TODO: Create an embedding layer of shape [vocab_size, rnn_size]
        #####
        self.embedding = nn.Embedding(vocab_size, rnn_size)
        #####

        #####
        # TODO: Create a RNN stack with `num_layers` layers with tanh
        #         nonlinearity. Between each layers, there is a dropout of
        #         `dropout`. Implement it with a *single* call to `torch.nn` APIs
        #####
        if model_type == 'lstm':
            self.rnn = nn.LSTM(
                input_size=rnn_size,
                hidden_size=rnn_size,
                num_layers=num_layers,
                dropout=dropout if num_layers > 1 else 0.0, # dropout only between layers
                batch_first=True # input/output shape: (batch, seq_len, features)
            )
        else:
```

```

        self.rnn = nn.RNN(
            input_size=rnn_size,
            hidden_size=rnn_size,
            num_layers=num_layers,
            nonlinearity='tanh',
            dropout=dropout if num_layers > 1 else 0.0,
            batch_first=True
        )

#####

#####
# TODO: Create a linear layer that maps from rnn_size to n_categories
#####
self.linear = nn.Linear(rnn_size, n_categories)
#####

```

Now for the helper function:

```

def letterToIndex(letter):
    """Map a letter (a character) into its index in all_letters string"""
    # all_letters contains: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ .,;'-
    return all_letters.find(letter)

```

Reasoning

Embedding Layer (`nn.Embedding`):

- Creates a lookup table of shape `(vocab_size, rnn_size)`
- Each character index gets mapped to a learnable vector of size `rnn_size`
- Input: `(batch, seq_len)` of integer indices → Output: `(batch, seq_len, rnn_size)`

RNN/LSTM Stack (`nn.RNN` or `nn.LSTM`):

- `input_size=rnn_size`: Matches embedding output dimension
- `hidden_size=rnn_size`: Hidden state size (also output dimension)
- `num_layers`: Stacks multiple RNN layers vertically
- `dropout`: Applied between layers (only effective when `num_layers > 1`)
- `batch_first=True`: Critical! Makes input/output shape `(batch, seq_len, features)` instead of `(seq_len, batch, features)`
- `nonlinearity='tanh'`: For vanilla RNN (LSTM has its own internal activations)

Linear Layer:

- Maps from hidden state dimension to number of classes
- Applied to the final hidden state to produce logits for classification

`letterToIndex` function:

- Uses Python's `str.find()` method to get the index of a character
- Returns the position of `letter` in `all_letters` (0-indexed)
- Returns -1 if not found (though all valid characters should be in `all_letters`)

Note on Forward Pass

The forward pass (not shown in the TODO) would typically:

1. Embed the input: `x = self.embedding(x)`
2. Run through RNN: `output, hidden = self.rnn(x)`
3. Take last hidden state and classify: `logits = self.linear(output[:, -1, :])`

Ready for the next task!

👤 Coding Task 3(a): Implement the decoder for an Autoencoder The Autoencoder has an encoder that maps input x to latent representation z , and a decoder that maps z back to reconstructed x' . The encoder is already implemented as:

```
def _build_encoder(self):
    layers = []
    prev_size = self.input_size
    for layer_id, size in enumerate(self.hidden_sizes):
        layers.append(nn.Linear(prev_size, size))
        if layer_id < len(self.hidden_sizes)-1:
            layers.append(self.activation_cls())
        prev_size = size
    return nn.Sequential(*layers)
```

Implement the decoder (which should be symmetric to the encoder):

```
def _build_decoder(self):
    layers = []
    #####
    # TODO: Implement the decoder. The decoder should be a mirror of the # encoder, but in reverse order. The last layer
    # should output # self.input_size dimensions. # # Hint: Use reversed(self.hidden_sizes) to iterate in reverse order
    #####
    # YOUR CODE HERE
    #####
    return nn.Sequential(*layers)
```

Thought process: Architected symmetric decoder by reversing encoder structure and logic.

The user wants me to implement the decoder for an autoencoder. The decoder should be a mirror of the encoder but in reverse order.

Looking at the encoder:

- It iterates through `self.hidden\_sizes`
- For each layer, it creates a Linear layer from `prev\_size` to `size`
- It adds an activation after each layer EXCEPT the last layer
- The final output size is the last element of `hidden\_sizes` (which is the latent dimension)

For the decoder:

- It should start from the latent dimension (last element of hidden_sizes)
- It should iterate through hidden_sizes in reverse
- The final output should be `self.input\_size`
- Activations should be added between layers but not after the final layer

Let me think about this more carefully. If hidden_sizes = [128, 64, 32]:

- Encoder: input_size -> 128 (+ activation) -> 64 (+ activation) -> 32 (no activation)
- Decoder should be: 32 -> 64 (+ activation) -> 128 (+ activation) -> input_size (no activation)

So for the decoder:

- reversed(hidden_sizes) = [32, 64, 128]
- We need to map: 32 -> 64, 64 -> 128, 128 -> input_size

Let me trace through:

- Start with prev_size = hidden_sizes[-1] = 32 (the latent size)
- reversed_sizes = [32, 64, 128]
- We want to skip the first one and use it as starting point, then map to subsequent ones
- Or we iterate through reversed list starting from index 1

Actually, let me re-read the encoder:

```
python
prev_size = self.input_size
for layer_id, size in enumerate(self.hidden_sizes):
    layers.append(nn.Linear(prev_size, size))
    if layer_id < len(self.hidden_sizes)-1:
```

```
        layers.append(self.activation_cls())
    prev_size = size
    ...
```

I see the pattern now. The encoder adds layers and activations based on the layer index. For the decoder, I'll need to follow a similar logic but in reverse. The key is to track the layer index and apply activations conditionally, just like in the encoder implementation.

The decoder will transform the latent representation back through progressively larger layers, ultimately reconstructing the original input size. I'll need to carefully manage layer connections and activation functions to ensure symmetry with the encoder's architecture.

By reversing the hidden layer sizes and iterating through them, I can construct linear layers that progressively expand from the latent space. The key is maintaining activation functions between intermediate layers while leaving the final layer without activation to directly map to the input size.

The layer progression will look like: 32 -> 64 with activation, then 64 -> 128 with activation, and finally 128 -> input_size without activation. This mirrors the encoder's structure and ensures a balanced transformation back to the original input dimensions.

The implementation handles edge cases like single hidden layer sizes by dynamically building layers based on the reversed size list, ensuring flexibility in network architecture.

The first approach creates a straightforward decoder with a single linear layer when hidden sizes are minimal, mapping directly from the latent space to the input dimension without additional activations.

An alternative method more closely mirrors the encoder's structure by explicitly appending the input size and conditionally applying activations based on layer position. This approach provides a more symmetrical decoder construction that adapts to varying network complexities.

The code elegantly manages different network configurations, ensuring robust decoder generation across different hidden layer configurations.

Implementing the Autoencoder Decoder

The decoder should mirror the encoder structure but in reverse, mapping from the latent space back to the original input dimension.

Understanding the Structure

If the encoder has `hidden_sizes = [128, 64, 32]`:

- **Encoder:** `input_size → 128 (+ act) → 64 (+ act) → 32` (latent)
- **Decoder:** `32 → 64 (+ act) → 128 (+ act) → input_size` (reconstruction)

Key observations:

- The decoder starts from the latent dimension (last element of `hidden_sizes`)
- It goes through the hidden sizes in reverse order
- The final layer outputs `input_size` with **no activation** (to allow unconstrained reconstruction)

Implementation

```
def _build_decoder(self):
    layers = []

    #####
    # TODO: Implement the decoder. The decoder should be a mirror of the
    #       encoder, but in reverse order. The last layer should output
    #       self.input_size dimensions.
    #####

    reversed_sizes = list(reversed(self.hidden_sizes))
    prev_size = reversed_sizes[0] # Start from latent dimension

    # Intermediate layers with activations
    for size in reversed_sizes[1:]:
        layers.append(nn.Linear(prev_size, size))
        layers.append(self.activation_cls())
        prev_size = size

    # Final layer: map back to input_size (no activation)
    layers.append(nn.Linear(prev_size, self.input_size))
    #####
    return nn.Sequential(*layers)
```

Reasoning

Step-by-step breakdown:

- Reverse the hidden sizes:** `reversed_sizes = list(reversed(self.hidden_sizes))`
 - If `hidden_sizes = [128, 64, 32]`, then `reversed_sizes = [32, 64, 128]`
- Start from latent dimension:** `prev_size = reversed_sizes[0]`
 - This is the output of the encoder (the bottleneck)
- Build intermediate layers:** Loop through `reversed_sizes[1:]`
 - Each layer maps from previous size to next size
 - Add activation after each intermediate layer
- Final reconstruction layer:** `nn.Linear(prev_size, self.input_size)`
 - Maps back to original input dimension
 - No activation — allows the network to output any value range

Example trace with `hidden_sizes = [128, 64, 32]` and `input_size = 784`:

Step	Layer	Activation
1	Linear(32 → 64)	✓
2	Linear(64 → 128)	✓
3	Linear(128 → 784)	✗

This perfectly mirrors the encoder but in reverse!

Ready for the next task!

👤 Coding Task 3(a) continued: Implement training step for Denoising Autoencoder class

```

DenoisingAutoencoder(Autoencoder): def init(self, input_size, hidden_sizes, activation_cls=nn.ReLU, noise_std=0.5):
    super().init(input_size, hidden_sizes, activation_cls) self.noise_std = noise_std def train_step(self, x, optimizer):
    #####
    # TODO: Implement training step of the denoising autoencoder. # # Hint: Add a zero-mean i.i.d. gaussian noise of a
    standard deviation of # noise_std to the input of the encoder
    #####
    # YOUR CODE HERE
    #####
    loss = self.loss(x, x_hat) optimizer.zero_grad() loss.backward() optimizer.step() return loss

```

Thought process: Deconstructed denoising autoencoder training pipeline with noise injection strategy.

The user wants me to implement the training step for a Denoising Autoencoder. The key idea is:

1. Add Gaussian noise to the input `x` to create a corrupted version `x_noisy`
2. Pass `x_noisy` through the encoder to get latent representation `z`
3. Pass `z` through the decoder to get reconstruction `x_hat`
4. Compute loss between `x_hat` and the ORIGINAL clean `x` (not the noisy version)
5. Backpropagate and update

The noise should be zero-mean Gaussian with standard deviation ``self.noise_std``.

Implementing the Denoising Autoencoder Training Step

The key idea of a denoising autoencoder is to corrupt the input with noise, then train the model to reconstruct the *original* clean input.

Understanding Denoising Autoencoders

1. **Add noise:** Corrupt input `x` → `x_noisy`
2. **Encode:** `x_noisy` → `z` (latent representation)
3. **Decode:** `z` → `x_hat` (reconstruction)
4. **Loss:** Compare `x_hat` to original **clean** `x` (not the noisy version!)

This forces the network to learn robust features that can recover the true signal from corrupted input.

Implementation

```

def train_step(self, x, optimizer):
    #####
    # TODO: Implement training step of the denoising autoencoder.
    #
    # Hint: Add a zero-mean i.i.d. gaussian noise of a standard deviation of
    #       `noise_std` to the input of the encoder
    #####

```

```

# Create noisy input by adding Gaussian noise
# torch.randn_like creates a tensor of same shape with values from N(0,1)
# Multiply by noise_std to get N(0, noise_std^2)
noise = torch.randn_like(x) * self.noise_std
x_noisy = x + noise

# Forward pass: encode noisy input, decode to reconstruction
z = self.encoder(x_noisy)
x_hat = self.decoder(z)
#####
loss = self.loss(x, x_hat)
optimizer.zero_grad()
loss.backward()
optimizer.step()
return loss

```

Reasoning

Adding Gaussian noise:

- `torch.randn_like(x)`: Creates a tensor with the same shape and device as `x`, filled with samples from standard normal distribution $N(0, 1)$
- Multiplying by `self.noise_std` scales the distribution to $N(0, \text{noise_std}^2)$
- Adding to `x` gives us: `x_noisy = x + ε`, where $\epsilon \sim N(0, \text{noise_std}^2)$

Forward pass:

- `self.encoder(x_noisy)`: Encode the *corrupted* input
- `self.decoder(z)`: Decode back to reconstruction

Loss computation:

- `self.loss(x, x_hat)`: Compare reconstruction to the *original clean* `x`
- This is crucial—we want the network to learn to denoise!

Why denoising helps:

- Prevents the autoencoder from simply learning the identity function
- Forces the model to learn meaningful features that capture the underlying structure of the data
- Acts as a form of regularization
- Makes the learned representations more robust to small perturbations

Ready for the next task!

👤 Coding Task 3(a) continued: Implement training step for Masked Autoencoder class MaskedAutoencoder(Autoencoder):

```
def init(self, input_size, hidden_sizes, activation_cls=nn.ReLU, mask_prob=0.25): super().init(input_size, hidden_sizes,
activation_cls) self.mask_prob = mask_prob
def train_step(self, x, optimizer):
#####
# TODO: Implement training step of the masked autoencoder. # # Hint: Generate a mask with i.i.d. probabilities of
mask_prob for each # entry, and apply it to the input of the encoder, setting to zero # the entries where the mask is
activated.
#####
# YOUR CODE HERE
#####
loss = self.loss(x, x_hat) optimizer.zero_grad() loss.backward() optimizer.step() return loss
```

Thought process: Architected masked input strategy for autoencoder training implementation.

The user wants me to implement the training step for a Masked Autoencoder. Similar to the denoising autoencoder, but instead of adding noise, we randomly mask (set to zero) some entries of the input with probability `mask\_prob`.

The key steps:

1. Generate a binary mask where each entry is 1 with probability (1 - mask_prob) and 0 with probability mask_prob
2. Apply the mask to the input (element-wise multiplication)
3. Pass the masked input through encoder and decoder
4. Compute loss between reconstruction and original clean input

Implementing the Masked Autoencoder Training Step

Similar to the denoising autoencoder, but instead of adding noise, we randomly "drop out" (mask to zero) some input features.

Understanding Masked Autoencoders

1. **Create mask:** Randomly select entries to zero out with probability `mask_prob`
2. **Apply mask:** `x_masked = x * mask` (zeroing selected entries)
3. **Encode & Decode:** `x_masked` → `z` → `x_hat`
4. **Loss:** Compare `x_hat` to original **clean** `x`

This forces the network to learn to "fill in" missing information.

Implementation

```
def train_step(self, x, optimizer):
#####
# TODO: Implement training step of the masked autoencoder.
#
# Hint: Generate a mask with i.i.d. probabilities of mask_prob for each
```

```

#         entry, and apply it to the input of the encoder, setting to zero
#         the entries where the mask is activated.
#####
# Generate binary mask: 1 = keep, 0 = mask out
# torch.rand_like gives uniform values in [0, 1)
# Values >= mask_prob become 1 (keep), values < mask_prob become 0 (drop)
mask = (torch.rand_like(x) >= self.mask_prob).float()

# Apply mask to input (element-wise multiplication)
x_masked = x * mask

# Forward pass: encode masked input, decode to reconstruction
z = self.encoder(x_masked)
x_hat = self.decoder(z)
#####
loss = self.loss(x, x_hat)
optimizer.zero_grad()
loss.backward()
optimizer.step()
return loss

```

Reasoning

Generating the mask:

- `torch.rand_like(x)`: Creates tensor of same shape with uniform random values in [0, 1)
- `>= self.mask_prob`: Creates boolean tensor where `True` means "keep this entry"
- `.float()`: Converts to float tensor (1.0 = keep, 0.0 = mask)

Example with `mask_prob = 0.25`:

- ~75% of entries will have random value $\geq 0.25 \rightarrow \text{mask} = 1$ (keep)
- ~25% of entries will have random value $< 0.25 \rightarrow \text{mask} = 0$ (drop)

Applying the mask:

- `x_masked = x * mask`: Element-wise multiplication zeros out masked entries

Loss computation:

- Compare reconstruction to original **clean** `x`
- The model must learn to reconstruct even the masked entries

Comparison: Denoising vs Masked

Aspect	Denoising AE	Masked AE
Corruption	Add Gaussian noise	Zero out entries
Input range	Can exceed original range	Stays within [0, x]
Information	All info present (but noisy)	Some info completely removed
Task	Filter out noise	Fill in missing data

Both techniques prevent the autoencoder from learning a trivial identity mapping and encourage learning robust, meaningful representations.

Ready for the next task!

👤 Coding Task 5(c): Implement adjacency matrix computation for graph clustering We interpret each data point as a node in a graph. The edge connection between two vertices is determined by weighted distances: $A_{\{i,j\}} = e^{-\gamma ||x_i - x_j||^2}$ where x_i and x_j are data points and γ is positive. `def get_adjacency_matrix(gamma, X):` """ Inputs: - gamma: positive scalar coefficient - X: data matrix of shape (n_samples, n_features)

```
Returns:
- adjacency_matrix: matrix of shape (n_samples, n_samples)
"""
# TODO: fill in your code here
# Hint: You may find scipy.spatial.distance.cdist useful
# adjacency_matrix = ?
return adjacency_matrix
```

Is this a directed or undirected graph?

Thought process: Deconstructed kernel formula and determined graph symmetry properties.

The user is asking me to implement the adjacency matrix computation for graph clustering using a Gaussian/RBF kernel. Let me think through this:

1. The formula is $A_{\{i,j\}} = e^{-\gamma ||x_i - x_j||^2}$
2. This is essentially computing pairwise squared Euclidean distances, then applying the Gaussian kernel

For the implementation:

- Use `scipy.spatial.distance.cdist` to compute pairwise distances
- Square the distances (or use 'sqeuclidean' metric directly)
- Apply the exponential transformation

For the directed vs undirected question:

- Since $||x_i - x_j||^2 = ||x_j - x_i||^2$, we have $A_{\{i,j\}} = A_{\{j,i\}}$
- This means the adjacency matrix is symmetric, so it's an undirected graph

Implementing Adjacency Matrix for Graph Clustering

This computes a similarity-based adjacency matrix using a Gaussian (RBF) kernel.

Understanding the Formula

$$A_{i,j} = e^{-\gamma ||x_i - x_j||^2}$$

- When points are **close**: distance is small → weight is close to 1 (strong connection)
- When points are **far**: distance is large → weight is close to 0 (weak connection)

Implementation

```
def get_adjacency_matrix(gamma, X):
    """
    Inputs:
    - gamma: positive scalar coefficient
    - X: data matrix of shape (n_samples, n_features)

    Returns:
    - adjacency_matrix: matrix of shape (n_samples, n_samples)
    """
    from scipy.spatial.distance import cdist

    # Compute pairwise squared Euclidean distances
    # cdist with 'sqeuclidean' computes ||x_i - x_j||^2 for all pairs
    squared_distances = cdist(X, X, metric='sqeuclidean')

    # Apply Gaussian kernel: A_{i,j} = exp(-gamma * ||x_i - x_j||^2)
    adjacency_matrix = np.exp(-gamma * squared_distances)

    return adjacency_matrix
```

Reasoning

Using `cdist`:

- `cdist(X, X, metric='sqeuclidean')` computes the squared Euclidean distance between every pair of rows
- Returns a matrix of shape `(n_samples, n_samples)`
- Entry `[i, j]` contains `||x_i - x_j||^2`

Applying the kernel:

- Element-wise: `np.exp(-gamma * squared_distances)`
- Transforms distances into similarities

Is this a Directed or Undirected Graph?

This is an undirected graph.

Proof: The adjacency matrix is symmetric because:

$$A_{i,j} = e^{-\gamma \|x_i - x_j\|^2} = e^{-\gamma \|x_j - x_i\|^2} = A_{j,i}$$

The Euclidean distance is symmetric: $\|x_i - x_j\| = \|x_j - x_i\|$

Properties of this adjacency matrix:

Property	Value
Symmetric	Yes ($A = A^T$)
Diagonal	All 1s (distance to self is 0, $e^0 = 1$)
Value range	$(0, 1]$
Graph type	Undirected, weighted, fully connected

This is commonly used in spectral clustering, where the Gaussian kernel creates a "soft" neighborhood structure based on similarity.

Ready for the next task!

👤 Coding Task 5(d): Implement degree matrix computation The degree matrix of an undirected graph is a diagonal matrix containing the degree of each vertex: $D_{\{i,j\}} = \text{deg}(v_i)$ if $i = j$, else 0 where $\text{deg}(v_i)$ counts the number of edges attached to vertex i .
def get_degree_matrix(adjacency_matrix): """ Inputs: - adjacency_matrix: matrix of shape (n_samples, n_samples)

```
Returns:
- degree_matrix: diagonal matrix of shape (n_samples, n_samples)
"""
# TODO: fill in your code here
# degree_matrix = ?
return degree_matrix
```

Thought process: Formulated degree computation strategy using row summation.

The user is asking me to implement the degree matrix computation for graph clustering.

For a weighted graph (which is what we have since our adjacency matrix contains continuous values from the Gaussian kernel), the degree of a vertex is typically defined as the sum of the weights of all edges connected to it.

So for vertex i :
 $\text{deg}(v_i) = \sum_j A_{\{i,j\}}$

This is the sum of row i (or column i , since the matrix is symmetric).

The degree matrix D is a diagonal matrix where $D_{\{i,i\}} = \text{deg}(v_i)$.

I can compute this by:

1. Sum each row of the adjacency matrix to get the degrees
2. Create a diagonal matrix from these degrees

Using numpy, this would be:

```
```python
degrees = np.sum(adjacency_matrix, axis=1) # Sum along rows
degree_matrix = np.diag(degrees)
```
```

Or alternatively:

```
```python
degree_matrix = np.diag(adjacency_matrix.sum(axis=1))
```
```

Implementing the Degree Matrix for Graph Clustering

The degree matrix captures how "connected" each node is in the graph.

Understanding the Degree Matrix

For a **weighted graph** (like ours with Gaussian kernel weights), the degree of a vertex is the **sum of edge weights** connected to it:

$$\deg(v_i) = \sum_j A_{i,j}$$

The degree matrix is then a diagonal matrix:

$$D_{i,j} = \begin{cases} \deg(v_i) = \sum_k A_{i,k} & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

Implementation

```
def get_degree_matrix(adjacency_matrix):  
    """  
    Inputs:  
    - adjacency_matrix: matrix of shape (n_samples, n_samples)  
  
    Returns:  
    - degree_matrix: diagonal matrix of shape (n_samples, n_samples)  
    """  
    # Sum each row to get the degree of each vertex  
    degrees = np.sum(adjacency_matrix, axis=1)  
  
    # Create diagonal matrix from the degrees  
    degree_matrix = np.diag(degrees)  
  
    return degree_matrix
```

Reasoning

Step 1 - Compute degrees:

- `np.sum(adjacency_matrix, axis=1)` sums along rows
- For vertex i , this computes $\sum_j A_{i,j}$
- Returns a 1D array of shape `(n_samples,)`

Step 2 - Create diagonal matrix:

- `np.diag(degrees)` creates a 2D matrix with `degrees` on the diagonal and zeros elsewhere
- Returns shape `(n_samples, n_samples)`

Example

For a small adjacency matrix:

```
A = [[1.0, 0.8, 0.2],  
      [0.8, 1.0, 0.5],  
      [0.2, 0.5, 1.0]]  
  
degrees = [2.0, 2.3, 1.7] # Row sums
```

```
D = [[2.0, 0.0, 0.0],  
     [0.0, 2.3, 0.0],  
     [0.0, 0.0, 1.7]]
```

Why the Degree Matrix Matters

The degree matrix is used to compute the **graph Laplacian**:

- Unnormalized Laplacian: $L = D - A$
- Normalized Laplacian: $L_{norm} = I - D^{-1/2}AD^{-1/2}$

The Laplacian's eigenvectors are used in spectral clustering to find cluster assignments!

Ready for the next task!

👤 Coding Task 5(g): Convert to stochastic matrix The symmetrically normalized adjacency matrix M can be viewed as a transition matrix in a Markov Chain. To fully interpret it as transition probabilities, it needs to be a proper stochastic matrix where columns sum to 1. `def stochastic_matrix_converter(M):` """ Inputs: - M : symmetrically normalized adjacency matrix of shape (n, n)

```
Returns:
- M_stoch: stochastic version where each column sums to 1
"""
# TODO: fill in your code here
# M_stoch = ?
return M_stoch
```

Thought process: Devised column normalization strategy for stochastic conversion.

The user wants me to implement a function that converts a symmetrically normalized adjacency matrix into a stochastic matrix where each column sums to 1.

A stochastic matrix (specifically a column-stochastic or left-stochastic matrix) has the property that each column sums to 1, representing transition probabilities in a Markov chain.

To convert matrix M to a column-stochastic matrix, we need to normalize each column by its sum:

```
M_stoch[:, j] = M[:, j] / sum(M[:, j])
```

This can be done efficiently by dividing each column by its sum.

Implementing Stochastic Matrix Conversion

A stochastic matrix represents transition probabilities in a Markov chain, where each column must sum to 1 (for a column-stochastic matrix).

Understanding Stochastic Matrices

For a **column-stochastic matrix**:

- Each column represents the transition probabilities from a given state
- All entries are non-negative
- Each column sums to 1: $\sum_i M_{i,j} = 1$ for all j

To convert matrix M to column-stochastic form, we normalize each column by its sum:

$$M_{i,j}^{stoch} = \frac{M_{i,j}}{\sum_k M_{k,j}}$$

Implementation

```
def stochastic_matrix_converter(M):
    """
```

```

Inputs:
- M: symmetrically normalized adjacency matrix of shape (n, n)

Returns:
- M_stoch: stochastic version where each column sums to 1
"""

# Compute the sum of each column
column_sums = np.sum(M, axis=0) # Shape: (n,)

# Divide each column by its sum (broadcasting handles this)
M_stoch = M / column_sums

return M_stoch

```

Reasoning

Step 1 - Compute column sums:

- `np.sum(M, axis=0)` sums along rows (down each column)
- Returns a 1D array of shape `(n,)` containing the sum of each column

Step 2 - Normalize columns:

- `M / column_sums` broadcasts the division
- Each element `M[i, j]` is divided by `column_sums[j]`
- Result: each column now sums to 1

Example

```

M = [[0.4, 0.3, 0.1],
      [0.2, 0.6, 0.2],
      [0.2, 0.3, 0.1]]

column_sums = [0.8, 1.2, 0.4]

M_stoch = [[0.50, 0.25, 0.25],
            [0.25, 0.50, 0.50],
            [0.25, 0.25, 0.25]]

# Verify: each column sums to 1.0 ✓

```

Connection to Markov Chains

In the context of graph clustering:

- $M_{i,j}^{stoch}$ represents the probability of transitioning from node j to node i
- A random walk on the graph follows these transition probabilities
- Clusters emerge as regions where the random walker tends to stay (high internal transition probabilities)

The stationary distribution of this Markov chain relates to the cluster structure of the graph!

Ready for the next task!